SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET
PROGRAMMING COURSE CODE: CSA43

COURSE OUTCOME COVERED

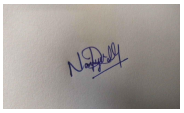
CO-1 : Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool : Application-Based Questions

Weightage: 40%

Code of Conduct:

I Narravula Navya Sri - 192324220 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*



Signature of the student:

QUESTIONS: **Total Marks: 30**

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**

1. Identify missing HTML5 semantic elements

The form lacks a complete HTML5 document structure and semantic elements such as:

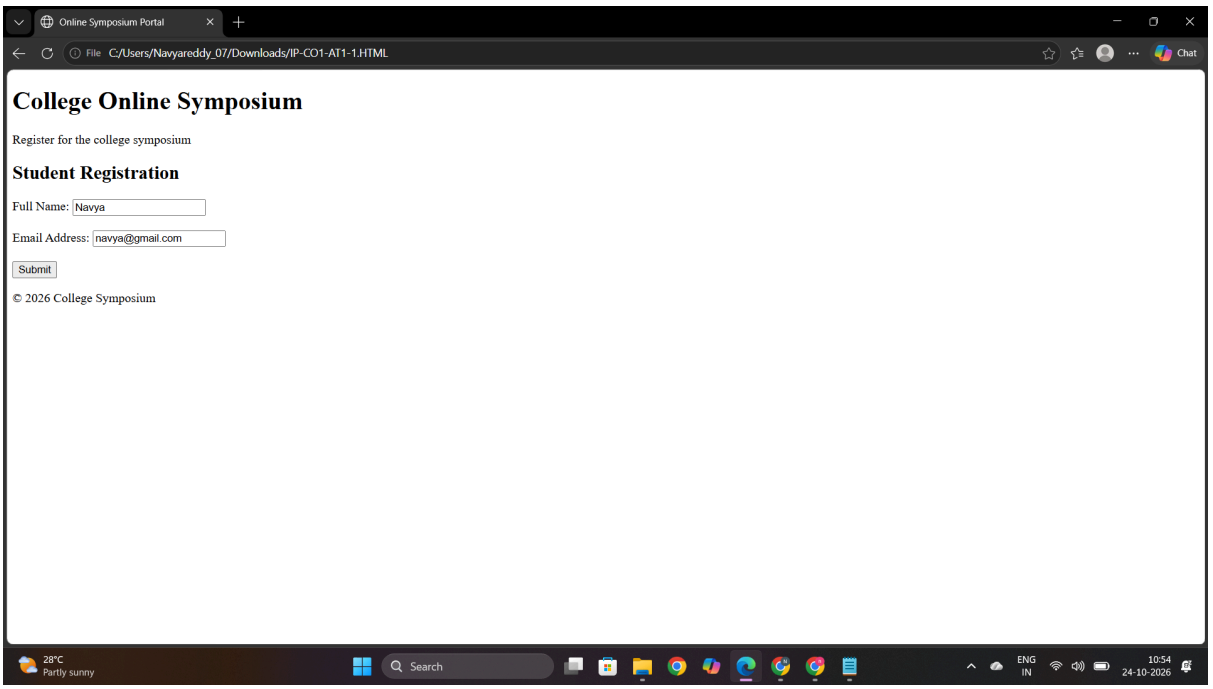
- `<header>` — for the page heading.
- `<main>` — for the main content.
- `<section>` — to group symposium registration content.
- `<footer>` — for copyright or contact details.

It also needs a `<label>` for each input to improve accessibility.

2. Modify the structure to make it standards-compliant

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Online Symposium Registration</title>
</head>
<body>
  <header>
    <h1>College Online Symposium</h1>
  </header>
  <main>
    <section>
      <h2>Student Registration</h2>
```

```
        <form action="/register" method="POST">
            <label for="name">Full Name:</label>
            <input type="text" id="name" name="name" required>
            <br><br>
            <label for="email">Email Address:</label>
            <input type="email" id="email" name="email" required>
            <br><br>
            <button type="submit">Register</button>
        </form>
    </section>
  </main>
  <footer>
    <p>&copy; 2026 College Symposium</p>
  </footer>
</body>
</html>
```



3. Interpret the HTTP request generated during form submission

When the student submits the form, the browser sends an HTTP `POST` request to `/register`.

Example request:

POST /register HTTP/1.1
Host: example.com
Content-Type: application/x-www-form-urlencoded

name=Ravi+Kumar&email=ravi%40example.com

- POST: Sends form data in the request body.
- Action: Specifies the server endpoint `/register`.
- Content-Type: Identifies the format of the submitted data.

- Request body: Contains the values of the named form fields.

4. Analyze the HTTP response cycle

The server may return:

- `200 OK` — request processed successfully.
- `201 Created` — registration resource created.
- `400 Bad Request` — invalid submitted data.
- `500 Internal Server Error` — unexpected server failure.

The browser processes the response and displays the resulting page or message.

5. Improvements for usability and accessibility

- Use `<label>` elements associated with input IDs.
- Add the `required` attribute to mandatory fields.
- Use appropriate input types, such as `email`.
- Provide clear validation and error messages.
- Ensure keyboard navigation and visible focus indicators.
- Use HTTPS to protect data in transit.
- Validate all submitted data on the server.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

1. Identify improper nesting and missing closing tags

Original code:

```
<html>
<head>
<title>Company</title>
</head>
<body>
<div>
<h1>Welcome
<p>About us
</div>
</body>
</html>
```

Problems:

- Missing `<!DOCTYPE html>`.
- Missing `lang` attribute on `<html>`.
- Missing character encoding and viewport metadata.
- Missing closing tags for `<h1>` and `<p>`.
- Generic `<div>` is used instead of meaningful semantic elements.

2. Analyze how different browsers may interpret this structure

Modern browsers use HTML parsing rules to recover from many markup errors. They may automatically close certain elements or construct a DOM tree even when closing tags are missing.

However, invalid or ambiguous markup can cause:

- Differences in the resulting DOM tree.
- Unexpected spacing and layout.
- CSS selectors behaving differently than intended.
- Accessibility tools interpreting the content incorrectly.

Browsers may recover differently depending on the type and location of the markup error.

3. Identify missing semantic elements

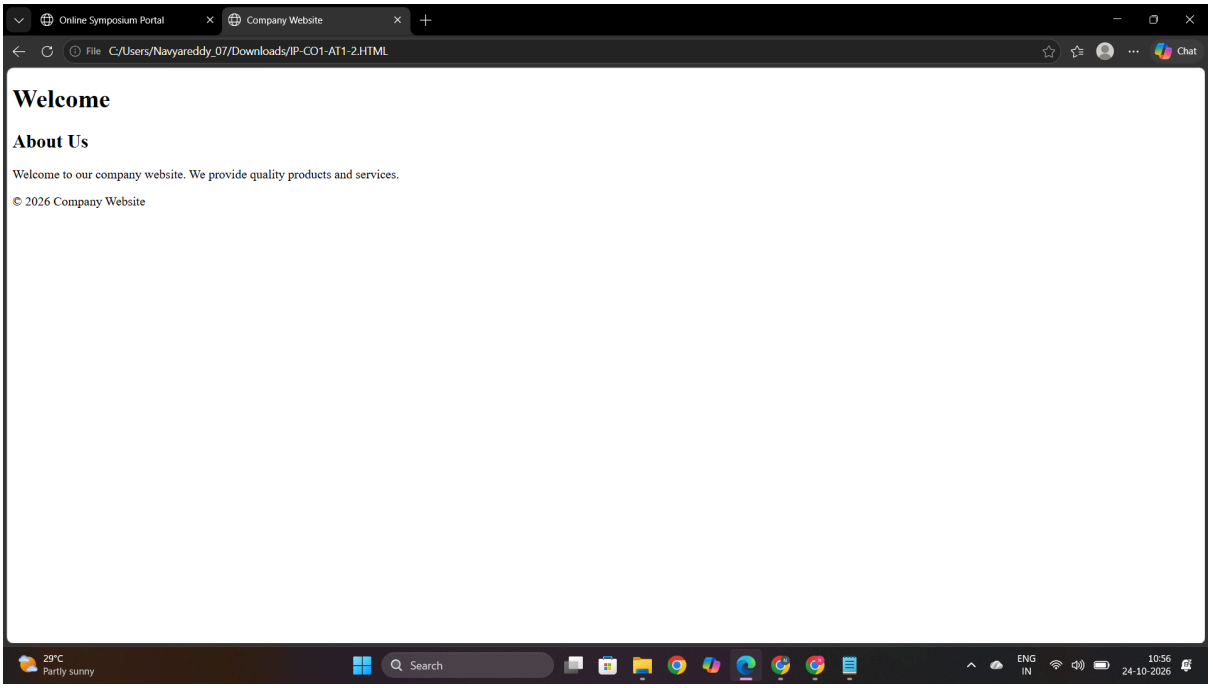
- `<header>` — identifies the page header.
- `<main>` — identifies the primary content.
- `<section>` — groups related content.
- `<footer>` — identifies footer information.

Semantic elements improve document structure and accessibility, but do not automatically guarantee cross-browser consistency.

4. Corrected W3C-compliant HTML structure

```
<!DOCTYPE html>
```

```
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company Website</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
  </header>
  <main>
    <section>
      <h2>About Us</h2>
      <p>Learn more about our company and services.</p>
    </section>
  </main>
  <footer>
    <p>&copy; 2026 Company Website</p>
  </footer>
</body>
</html>
```



5. Techniques to ensure cross-browser compatibility

1. Use valid HTML5 syntax and properly close elements.
2. Validate markup using the W3C Nu HTML Checker.
3. Use CSS resets or a consistent base stylesheet.
4. Prefer standard CSS properties and avoid unsupported features.
5. Test in Chrome, Firefox, Safari, and Edge.
6. Use responsive design with flexible layouts and media queries.
7. Check keyboard navigation, accessibility, and mobile layouts.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
   <input type="text" name="username">
   <input type="password" name="password">
   <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

1. Identify issues in the form configuration

Original code:

```
<form action="/submit" method="GET">
   <input type="text" name="username">
   <input type="password" name="password">
   <button type="button">Login</button>
</form>
```

The main problem is:

```
<button type="button">Login</button>
```

A button with type="button" does not submit the form by default. Therefore, clicking it does not trigger the browser's normal form submission.

2. Analyze why the HTTP request is not triggered

- The button is not a submit button.
- No JavaScript event handler is provided to submit the form.
- Consequently, clicking Login does not automatically send a request to /submit.

3. Interpret the role of the GET method

The GET method sends form data as URL query parameters.

For example:

```
GET /submit?username=ravi&password=Example123 HTTP/1.1
```

This is inappropriate for login credentials because the password may appear in the URL, browser history, server logs, and other systems that record URLs.

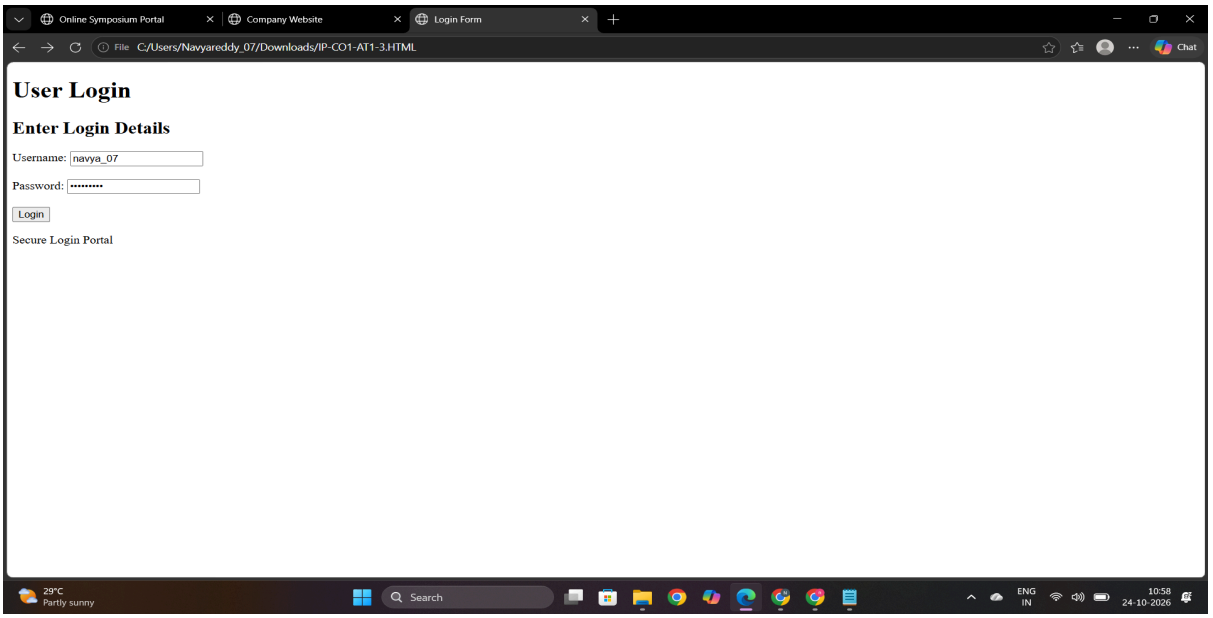
4. Corrected implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Login Form</title>
</head>
<body>
   <h2>User Login</h2>
   <form action="/submit" method="POST">
      <label for="username">Username:</label>
      <input type="text" id="username" name="username" required>
      <br><br>
      <label for="password">Password:</label>
      <input type="password" id="password" name="password" required>
      <br><br>
      <button type="submit">Login</button>
   </form>
</body>
</html>
```

How it works:

1. The user enters a username and password.
2. The required attributes trigger basic browser-side validation.
3. The submit button initiates form submission.
4. The browser sends a POST request to /submit.
5. The server verifies the credentials and returns an appropriate response.

This example assumes the server has a /submit endpoint that handles login requests.



5. Improvements for secure data transmission

- Use HTTPS to encrypt data during transmission.
- Use POST rather than GET for login credentials.
- Verify credentials on the server; never trust client-side validation alone.
- Store passwords using a suitable password-hashing algorithm, such as Argon2id, rather than plaintext.
- Use secure session cookies with HttpOnly, Secure, and appropriate SameSite settings.
- Implement rate limiting and generic authentication error messages.
- Never place passwords in URLs or log them unnecessarily.

Conclusion: Correct HTML structure, semantic elements, proper form submission, and secure HTTP practices help create accessible, reliable, and secure web applications.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps

Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand